

GOD's Vision

Technical & Business Documentation

An independent technical review of a self-hosted, all-in-one financial and world-data terminal — architecture, feature scope, business model considerations, and a candid status report on what currently works.

Prepared for	HealthIQ (operations@myhealthiq.io)
Subject	GOD's Vision — Next.js 14 financial/world-data terminal
Version reviewed	v2.0 codebase (per FEATURES_BUILT.md), reviewed July 8, 2026
Review method	Full static code review of the repository (app/, lib/, components/, prisma/, config). Live functional testing of localhost:3001 was to be performed jointly with the project owner.

Table of Contents

1. Overview & Purpose
2. Architecture
3. How It Works — Core User Flows
4. Why It Was Built — Problem & Market Gap
5. Scope — Current vs. Planned Features
6. Pricing & Business Model
7. Technical Feasibility Assessment
8. Status Report — What Works, What's Broken
9. Recommendations

1. Overview & Purpose

GOD's Vision is a self-hosted, browser-based financial and world-data terminal built as a personal/small-team alternative to professional market-data platforms. It runs as a single Next.js 14 application on port 3001 and presents itself as a dense, keyboard-navigable, dark-themed "terminal" interface in the visual tradition of Bloomberg Terminal — but assembled entirely from free or freemium public data APIs rather than licensed institutional data feeds.

The application aggregates 30+ distinct modules into one workspace: equities, crypto, forex, bonds/yield curves, commodities, options (with Black-Scholes Greeks), a portfolio tracker with real risk analytics (Sharpe, Sortino, VaR, beta), an AI-generated daily market brief, insider-trading feeds, a correlation heatmap, central-bank policy tracking, an economic calendar, a strategy backtester, and a collection of non-financial "situational awareness" panels — live flight tracking, earthquake/disease/weather feeds, ISS position, sports scores, and public webcams. A dedicated India Mode reformats a subset of these modules around Indian markets, currency, and RBI macro data.

Generative AI (Google Gemini 2.5 Flash) is woven in as an assistive layer rather than the product's core: it drafts the daily market brief, produces headline sentiment scores, and powers a narrative/analyst-note generator — each with a documented fallback to static content when the AI call fails or no API key is present.

In its current state, the product is best described as a **working prototype / personal analytics dashboard** rather than a shipped commercial product: authentication, a Postgres database, and Stripe billing scaffolding all exist in code, but billing and OAuth are not configured in this environment, and the plan-gating logic that would separate a free tier from a paid tier is currently disabled at the code level (detailed in Section 8).

2. Architecture

2.1 Tech Stack

Layer	Technology	Notes
Framework	Next.js 14.2.5 (App Router), React 18, TypeScript 5	Runs via <code>`next dev -p 3001`</code> / <code>`next start -p 3001`</code> — port is hardcoded into npm scripts
Styling / UI	Tailwind CSS 3.4, Radix UI primitives, custom <code>`cn()`</code> helper (shadcn/ui convention)	No full shadcn/ui generated component library is present — only 5 custom UI files exist (Panel, ProGate, ErrorBoundary, GlossaryTooltip, TickerLink); Radix is used more directly than through shadcn wrappers
State	Zustand, React Context (Theme, Mode, Session, Linked-Panels)	—
Database / ORM	PostgreSQL via Prisma 5.16, <code>`@vercel/postgres`</code>	Schema still documented as SQLite in SETUP.md — that doc is stale; the live schema targets Postgres
Auth	NextAuth 4.24 with Prisma adapter, JWT sessions, bcrypt credentials + optional Google OAuth	Google OAuth provider only loads if <code>`GOOGLE_CLIENT_ID`</code> / <code>`SECRET`</code> are set — currently absent from env

Layer	Technology	Notes
Billing	Stripe (server + <code>@stripe/stripe-js` client), webhook handler for subscription lifecycle</code>	<code>`STRIPE_SECRET_KEY`</code> , price ID, and webhook secret are absent from env in this deployment
AI provider	Google Generative AI SDK — Gemini 2.5 Flash (<code>`gemini-2.5-flash`</code> , temp 0.3, 2048 max tokens)	<code>`@anthropic-ai/sdk`</code> remains listed in package.json but is unused dead weight — no route imports it
Market/data clients	yahoo-finance2, axios, cheerio (HTML scraping), rss-parser	Yahoo Finance (unauthenticated v8/v10/v1 endpoints) is the dominant backbone for equities, financials, correlation, technicals, and search
Charts	lightweight-charts (candlesticks), Recharts, D3, react-leaflet (maps), hls.js (video)	—
Realtime	socket.io / socket.io-client, native WebSocket hooks (Alpaca, Binance)	Alpaca streaming keys are present as example values in FEATURES_BUILT.md — a secrets-hygiene issue worth fixing

2.2 Layered Architecture

Presentation layer — App Router pages under `app/*` (30 top-level route segments) render server/client component trees composed from three component families: `components/terminal` (chrome — top bar, nav, command palette, watchlist sidebar, alert banner), `components/panels` (data widgets — market overview, crypto, news, weather, analyst, FII/DII flow, Nifty heatmap), and `components/modules` (standalone split-view variants used by the Linked-Panels split-screen feature).

API / backend layer — 60 route handlers under `app/api/*` act as a thin proxy/aggregation tier: each route fetches from one or more third-party public APIs (or scrapes a public webpage), normalizes the response, applies an in-memory TTL cache (`lib/cache.ts`, 15s–4hr depending on data volatility), and returns JSON. Per-route HTTP cache-control headers are also set centrally in `next.config.js`. Following a documented "server hardening" pass, most routes return HTTP 200 with fallback/mock data rather than a 5xx error when an upstream call fails — a deliberate resilience choice, but one that also means real backend failures are largely invisible to the frontend and to monitoring.

Data layer — Prisma/PostgreSQL persists user accounts, sessions, watchlists (in two parallel schemas — see Section 8), portfolio holdings, price/news/earthquake alerts, a DB-backed generic cache table, and persisted AI sentiment results. This sits alongside, not instead of, the in-memory cache and a separate client-side (localStorage) price-alert system — three distinct caching/persistence mechanisms coexist.

External integrations — over 20 distinct third-party sources are wired in: Yahoo Finance, CoinGecko, Binance, DeFiLlama, alternative.me (Fear & Greed), ExchangeRate-API, FRED/St. Louis Fed, disease.sh, OpenSky Network, Open-Meteo, OpenWeatherMap, NOAA/weather.gov, USGS, SEC EDGAR, OpenInsider.com (scraped), TheSportsDB, OpenF1, ESPN's public API, ForexFactory's calendar feed, central-bank RSS feeds (Fed/ECB/BOE/BOC), Reddit's public JSON endpoints, TfL and Windy webcams, Open Notify (ISS), and roughly 30 news RSS feeds spanning US and India-focused sources. The large majority of these require no API key; a handful of optional keys (Alpha Vantage, FRED, EIA, NewsAPI, OpenWeather, AISstream, Alpaca) unlock higher rate limits or additional detail when configured.

2.3 Architecture Diagram (textual)

Component		Talks to
Browser (Next.js client components)	→	Next.js App Router pages (app/*)
Pages	→	components/terminal, panels, modules
Client fetches	→	app/api/* route handlers (60 routes)
Route handlers	→	In-memory TTL cache (lib/cache.ts)
Cache miss	→	External public APIs / scrapers (20+ sources) or Gemini 2.5 Flash
Auth/session/user data	→	NextAuth (JWT) → Prisma ORM → PostgreSQL
Billing events	→	Stripe Checkout/Portal/Webhook → Prisma User.plan

3. How It Works — Core User Flows

Sign-in & session. A visitor lands on the dashboard (`/`) without needing to sign in for most read-only modules. Authenticated-only areas (`/portfolio`, `/alerts`, `/settings` — the last of which is referenced by middleware but has no corresponding page, likely a leftover) redirect to `/auth/signin`, a terminal-styled page offering email/password credentials or Google OAuth (when configured). NextAuth issues a JWT session carrying the user's id and plan.

Daily dashboard session. The home dashboard assembles a market overview strip, crypto/news/weather/earthquake panels, and an AI-written Daily Brief on load, with a USA/India mode toggle changing the underlying tickers, currency, and macro data source.

Deep-dive research. From any ticker mention (via the fuzzy-search command palette, `Ctrl+K`, or a clickable `TickerLink`), a user can jump to a stock's dedicated deep-dive panel, its financials page, options chain (Black-Scholes Greeks computed client-side), technicals (RSI/MACD/SMA/EMA), and analyst commentary (Gemini-generated).

Portfolio & risk tracking. Users log holdings on `/portfolio`; the app computes Sharpe ratio, Sortino ratio, CAPM beta/alpha, max drawdown, VaR, and CVaR against historical price series pulled from Yahoo Finance.

Alerts. Two independent alerting systems exist: a server/DB-backed `PriceAlert`/`NewsAlert` model surfaced on `/alerts`, and a separate client-side `localStorage` alert checker (`lib/alertChecker.ts`) — the latter was documented mid-2026 as fully built but 'not yet wired into any UI'.

Backtesting. On `/backtest`, users run SMA-crossover, RSI overbought/oversold, or buy-and-hold strategies against historical data entirely client-side, with no persistence of results.

Ambient / TV mode. `/godmode` is a fullscreen, keyboard-controlled ambient display that auto-cycles six scenes (world map, market pulse, crypto matrix, BTC chart, earthquake watch, global news) every 30 seconds — intended for a second monitor or lobby-display use case.

Situational-awareness panels. Flights (OpenSky, Leaflet map), disease/COVID tracking (disease.sh), live cameras (curated YouTube embeds + TfL/Windy feeds), ISS position, and sports scores round out a non-financial "world awareness" layer distinct from the market modules.

Upgrade flow (as designed). `/pricing` presents Free vs. Pro tiers; a Stripe Checkout session is created via `/api/stripe/checkout`, and the webhook updates the user's `plan` field on completion/cancellation. As detailed in Section 8, the code paths that should enforce this gate currently do not — every user is treated as Pro regardless of billing state.

4. Why It Was Built — Problem & Market Gap

Professional market-data terminals — Bloomberg Terminal being the category-defining example — bundle real-time institutional-grade data, analytics, messaging, and execution into a single (famously expensive) workstation subscription, commonly cited in the \$24,000–\$30,000+ per user per year range, sold primarily to institutional desks. That price point and sales motion put comprehensive, cross-asset market monitoring out of reach for individual investors, students, small funds, and hobbyist traders, who are instead left stitching together free brokerage apps, a scattering of single-purpose websites (one for crypto, one for weather alerts relevant to commodities, one for economic calendars, one for options Greeks), and manual spreadsheet tracking.

GOD's Vision targets that gap directly: a single, keyboard-driven, always-on workspace that mimics the density and workflow feel of a professional terminal, built entirely on free/low-cost public data APIs instead of licensed feeds.

The trade-off is explicit and structural rather than accidental — the product cannot offer Bloomberg's real-time institutional depth, its execution/trading capability, its compliance-grade audit trails, or its cross-user messaging network (a core part of Bloomberg's actual value proposition is the closed communication layer between subscribers, which no free-data clone can replicate). What it can plausibly offer instead is breadth of casual/retail-relevant coverage (crypto, India markets, situational-awareness feeds like flights and earthquakes) that Bloomberg does not prioritize, at zero-to-low marginal data cost, in a modern web UI.

The India Mode work in particular suggests an intended underserved-market angle: Indian retail investors are significantly less likely to have Bloomberg/Refinitiv access than institutional US users, and dedicated Indian retail terminals (e.g., some broker-native tools) tend to be narrower in scope than what this app aggregates.

5. Scope — Current vs. Planned Features

5.1 Implemented Today

- 30+ market/data modules: equities, crypto, forex, bonds & yield curve, commodities, options (Black-Scholes), screener, earnings, economic calendar, macro dashboard, correlation heatmap, sector heatmap, insider trading (two parallel implementations), central bank tracker, portfolio & risk analytics, backtester, glossary (71 terms), watchlists (two parallel data models).
- Non-financial situational panels: flights, earthquakes, disease/COVID tracker, weather, ISS tracker, sports, live cameras.
- AI layer: Gemini 2.5 Flash-powered daily brief, headline sentiment scoring, narrative/analyst-note generation, with documented static fallbacks when AI is unavailable.
- India Mode: dedicated Indian indices, forex, commodities, macro (RBI rates), crypto, and news routes with INR formatting — though several core pages (markets, crypto, forex, macro, news, weather) remain US-only regardless of mode.
- Accounts & persistence: NextAuth credential + optional Google sign-in, Prisma/Postgres-backed user data, Stripe billing scaffolding, ambient "godmode" TV display, command palette, 4-theme system (dark, dark-contrast, light, India/saffron accent), Linked Panels split-screen view.

5.2 Built but Not Fully Wired / Known Incomplete

- Plan gating (`isPro()`, `requirePro()`) is stubbed to treat every user as Pro — the Free/Pro feature split exists only in UI copy, not in enforcement.
- Client-side `alertChecker.ts` alert engine was documented as complete but not connected to any UI trigger as of the most recent internal report reviewed.
- `/settings` is referenced by route middleware but has no corresponding page.
- Stripe billing and Google OAuth require env vars (`STRIPE_SECRET_KEY`, `STRIPE_PRO_PRICE_ID`, `STRIPE_WEBHOOK_SECRET`, `GOOGLE_CLIENT_ID/SECRET`) that are not present in this environment's `.env`/`.env.local` — both integrations are effectively inert until configured.
- Options chain data is largely unavailable in practice because Yahoo's options endpoint requires authenticated cookies the app does not have.
- Reddit sentiment and some India news sources (Moneycontrol, BQ Prime) fall back to mock/blocked data due to upstream throttling or 403s.

5.3 Apparent Planned / Aspirational Scope

Internal reports reference real-time WebSocket quote streaming (Alpaca) and India-mode parity across all pages as active or near-term work; no forward-looking roadmap document beyond the existing internal reports was found in the repository, so anything past current functionality should be treated as inferred rather than officially committed.

6. Pricing & Business Model Considerations

The codebase defines exactly two tiers in `lib/stripe.ts`: a Free plan and a Pro plan priced at \$29/month, gating AI analyst notes, India Mode, unlimited watchlists/alerts, the correlation matrix, insider-trading data, and the central-bank tracker behind Pro. As noted above, this gate is not currently enforced in code, so from a business-model standpoint the product today gives away 100% of its feature set at \$0 — the monetization layer exists structurally but is switched off.

Considerations for a real launch:

- **Cost structure is unusually favorable** for this category: nearly all data sources are free public APIs rather than licensed institutional feeds, so gross margin at scale is closer to a typical SaaS product than to a traditional market-data vendor whose largest cost line is data licensing.
- **Rate-limit and scraping fragility is a real cost risk**: several data sources are unauthenticated scrapes (OpenInsider) or public endpoints not designed for production load (Reddit JSON, Yahoo Finance's undocumented endpoints, OpenSky). At scale, these are likely to require paid tiers, IP rotation, or replacement — a cost the current architecture does not yet budget for.
- **\$29/month sits in a crowded bracket** — well below Bloomberg but roughly in line with or above many single-purpose tools (TradingView Pro, portfolio trackers, crypto dashboards) that a prospective customer might already use instead of switching to an all-in-one product; the pitch has to be consolidation value, not raw feature novelty.
- **No usage metering or seat-based logic exists** — pricing is flat per-account regardless of API call volume, which is fine at low scale but does not protect margins if a small number of users generate disproportionate scraping/API load.
- **India Mode as a distinct/lower-priced regional tier** could be a reasonable wedge market given lower existing terminal penetration there, but would require completing India-mode coverage across the pages currently documented as US-only (Section 5.2) before it's a credible standalone offering.

7. Technical Feasibility Assessment

7.1 What's Feasible As-Is

The core architecture — Next.js API routes as a thin aggregation/caching layer over public data sources, rendered into a component library of reusable panels — is a sound, conventional pattern and scales reasonably well for a low-to-moderate user base. The in-memory + DB-backed caching, combined with route-level HTTP cache headers, is an appropriate design for read-heavy, tolerant-of-staleness financial dashboards. Client-side computation (Black-Scholes, technical indicators, portfolio risk metrics) is implemented without external dependencies and appears mathematically sound (Wilder smoothing for RSI, Abramowitz-Stegun erf approximation for Black-Scholes, standard CAPM regression for beta/alpha).

7.2 Risks & Feasibility Concerns

- **Build-time safety is disabled**: `next.config.js` sets `typescript.ignoreBuildErrors: true` and `eslint.ignoreDuringBuilds: true`, so the production build does not actually enforce type-checking or linting — despite internal reports repeatedly claiming "0 TypeScript errors." This is a meaningful quality-control gap for anything beyond a personal project.

- **Dependence on unauthenticated/undocumented third-party endpoints** (Yahoo Finance's internal v8/v10/v1 endpoints, OpenInsider scraping, Reddit JSON) is the single largest technical-risk concentration: these can change or be blocked without notice, and the app has no formal SLA with any of them. The "return 200 with fallback data" hardening pass mitigates visible crashes but can mask silent data staleness from end users.
- **Two competing implementations for the same feature** (insider trading: ``/insider`` vs ``/insiders``; watchlists: ``Watchlist`` vs ``WatchlistGroup`/`WatchlistItem`` models) indicate schema/feature drift that should be reconciled before further building on top of either.
- **Secrets hygiene**: example/placeholder API keys and Stripe values are committed inline in ``FEATURES_BUILT.md``. Even as placeholders, this is a habit worth correcting before any real secret is ever added to a docs file.
- **Real-time claims vs. implementation**: Socket.io and native WebSocket hooks for Alpaca/Binance streaming exist, but Alpaca's real-time quote feature depends on API keys not present in this environment, and the extent of production-readiness for the streaming path was not verifiable from static review alone.
- **No automated test suite was found** in the repository (no ``__tests__``, ``*.test.ts``, or CI config observed) — quality assurance today relies entirely on manual review sessions recorded in the internal ``md`` reports.

8. Status Report — What's Working vs. Broken/Incomplete

Methodology note: This assessment is based on a full static review of the source code, configuration, Prisma schema, and seven internal progress reports authored between June 27 and July 4, 2026 (README.md, SETUP.md, COMPLETION_REPORT.md, FEATURES_BUILT.md, TEST_REPORT.md, DASHBOARD_FIXES.md, INDIA_MODE.md, POLISH_REPORT.md). Live interactive testing of the running instance at localhost:3001 was intended to accompany this review but was not completed in-session due to a browser-automation connectivity issue; findings below reflect code-level evidence and the project's own prior test reports rather than a fresh live walkthrough. Any items below should be spot-checked against the live app before being treated as final.

Area	Status	Evidence / detail
Core dashboard, market panels, crypto, forex, macro, glossary	● Working	Backed by well-established free APIs (Yahoo Finance, CoinGecko, ExchangeRate-API); no code-level defects found on review.
Portfolio risk analytics (Sharpe/Sortino/beta/VaR)	● Working	Standard, correctly-implemented formulas in lib/portfolio-analytics.ts.
Options chain (Black-Scholes Greeks)	● Partially broken	Greeks math is sound, but live options data is largely unavailable — Yahoo's options endpoint needs authenticated cookies the app doesn't have; page shows an 'unavailable' fallback.
Fear & Greed radar (gold/SPY scoring)	● Broken	TEST_REPORT.md documents a scoring-calibration bug causing the gauge to permanently read '100 / EXTREME FEAR' for 2026 gold prices; no later report confirms a fix.
Plan gating (Free vs. Pro enforcement)	● Broken	lib/stripe.ts isPro() unconditionally returns true; lib/planGate.ts requirePro() unconditionally returns false — every user is treated as Pro regardless of billing status.
Stripe billing	● Not configured	Checkout/portal/webhook code exists but STRIPE_SECRET_KEY, STRIPE_PRO_PRICE_ID, and STRIPE_WEBHOOK_SECRET are absent from env in this deployment.
Google OAuth sign-in	● Not configured	Provider only loads if GOOGLE_CLIENT_ID/SECRET are set; both are absent — only email/password credential sign-in is currently live.
Client-side price alert engine	● Incomplete	lib/alertChecker.ts is fully implemented but was documented as not wired into any UI trigger as of the last internal report reviewed.
/settings route	● Missing	Protected by middleware.ts matcher but no corresponding app/settings page exists in the repo.
Insider trading (two implementations)	● Duplicated	/insider (OpenInsider scrape) and /insiders (SEC EDGAR RSS) overlap in purpose with no doc reconciling which is canonical.
Watchlists (two data models)	● Duplicated	Legacy WatchlistGroup/WatchlistItem and newer Watchlist models coexist, served by separate /api/watchlist and /api/watchlists routes.

Area	Status	Evidence / detail
Reddit sentiment	● Mock fallback	Public Reddit JSON endpoint is throttled/unauthenticated; falls back to mock data per TEST_REPORT.md.
Some India news sources (Moneycontrol, BQ Prime)	● Blocked	Documented as returning HTTP 403 to server-side requests; INDIA_MODE.md notes NSE's own API is also Cloudflare-blocked server-side, requiring a Yahoo .NS-suffix workaround.
India Mode coverage	● Partial	INDIA_MODE.md explicitly lists /markets, /crypto, /forex, /macro, /news, /weather as not yet India-aware regardless of the mode toggle.
AI features (Daily Brief, sentiment, narratives)	● Working with fallback	Gemini 2.5 Flash integration is live and used consistently; all call sites catch failures and fall back to static/mock content rather than erroring.
Build-time type/lint enforcement	● Disabled	next.config.js sets typescript.ignoreBuildErrors and eslint.ignoreDuringBuilds to true, contradicting internal reports' '0 TypeScript errors' claims.

Recommended before treating any of the above as final: open the live instance and manually exercise sign-in, the dashboard, godmode, portfolio, both insider pages, options, and alerts — the project owner offered to do this and share console/network errors back, and that pass should supersede the code-only status above wherever it conflicts.

9. Recommendations

- Re-enable and test `isPro()`/`requirePro()` before any real launch — the monetization gate is currently a no-op.
- Reconcile the duplicate insider-trading pages and duplicate watchlist data models into single implementations.
- Fix or remove the Fear & Greed gold/SPY scoring formula given its documented permanent-EXTREME-FEAR bug.
- Wire the completed client-side alert checker into the UI, or remove it if the DB-backed PriceAlert model is the intended path.
- Turn build-time TypeScript/ESLint enforcement back on and clear the resulting errors before making further correctness claims in internal docs.
- Refresh SETUP.md (still describes SQLite) and remove committed placeholder secrets from FEATURES_BUILT.md.
- Complete India Mode parity across the six pages documented as still US-only, or scope India Mode explicitly as partial in user-facing copy.
- Before commercial launch, budget for the operational risk of unauthenticated scraped/undocumented endpoints (Yahoo internal APIs, OpenInsider, Reddit) — these are the most likely source of unplanned downtime at scale.